PHASE H — v2.1

Final implementation specification

Engineering spec — ship-from-this-PDF detail
Every file path, type signature, integration point, test assertion,
deployment command, and past-incident defense

POLYMARKET 5-minute BTC up/down bot · 2026-05-13

Scope: a different engineer (or future-you) should be able to ship v2.1 from this document alone. Each section is grounded in actual code (read in /src/polymarket_bot/) + Obsidian memory (14 phase checkpoints) + Phase G analytical findings. Past-incident defenses are explicit.

Phase H deployment timeline:
H.1 strategic plan PDF ✓ (done today)
H.1.5 THIS SPEC (now)
H.2 code (next session, ~1h)
H.3 tests (next session, ~30min)
H.4 register + .envrc (next session)
H.5 VPS deploy + verify (next session)
H.6 7-day soak gate → PROMOTE / EXTEND / ROLLBACK

1. Executive summary

v2.1 is v2 + 4 filter changes (CH1-CH4) + Kelly sizing (CH5-CH6) + multi-fire (CH7-CH8). It exploits the Phase G winning cell (tier 30-50% × |Δ|<5, n=83, +$0.65/trade, 48% WR with asymmetric payoff that v2 currently REJECTS) while eliminating the two confirmed loser sub-groups (Δ=0 and EV ∈ [0.02, 0.05)).

The big architectural twist this spec adds vs the H.1 PDF: Kelly sizing requires live bankroll tracking that doesn't exist in PaperRunner today (bankroll is computed dashboard-side from settled records). And multi-fire requires lifting the singular `pending_decision` invariant in PaperRunner, not just the strategy's `_fired_for_window`. Both changes are scoped + tested below.

Files touched in v2.1 ship:

File	Action	Why
src/polymarket_bot/strategies/spot_aware_v2_1.py	NEW	the new strategy
src/polymarket_bot/strategies/kelly_config.py	NEW	KellyConfig dataclass
src/polymarket_bot/_paper/runner.py	EDIT	bankroll tracker + multi-fire support
scripts/run_paper.py	EDIT	strategy dispatch + bankroll-getter wire
tests/strategies/test_spot_aware_v2_1.py	NEW	~18 tests
tests/strategies/test_kelly_config.py	NEW	Kelly math tests
tests/paper/test_runner_multi_fire.py	NEW	PaperRunner multi-fire regression
design/spot_aware_v2_1.md	NEW	locked design doc (anti-vibe rule #1)

2. Code architecture

2.1 Class hierarchy

```
Strategy (Protocol)                                  [protocol.py]
  ■■ SpotAwareStrategy (v1, 'spot_aware')           [spot_aware.py]
       ■■ SpotAwareStrategyV2 ('spot_aware_v2')    [spot_aware_v2.py]
            ■■ SpotAwareStrategyV3                  [spot_aware_v3.py]
            ■   (v2 + regime gate; SHELVED post-Phase F)
            ■■ SpotAwareStrategyV2_1  ★ NEW         [spot_aware_v2_1.py]
                (v2 + Kelly + multi-fire + new filters)
```

v2.1 INHERITS from v2 (not from v3). Reason: v3's regime gate is optional in v2.1 (defaults OFF per Q2). If operator enables it later, v2.1 will compose RegimeDetector + super.on_event in the v3 style.

2.2 Event/decision flow

```
Feed (WS)         → MarketEvent
                        ■
PaperRunner._on_event ■→ strategy.on_event(event) ■■→ Decision | None
                        ■
                        ↓ (Decision returned)
              _on_decision(decision)
                        ■
                        ↓
              ShadowBook.attempt_fill        (taker simulation)
              DecisionLog.record_fired       (JSONL append)
              _state.record_decision         (singular pending_decision today)
                        ■
                        ↓ (window close)
              _settle_current_window         (gamma → chainlink fallback)
              DecisionLog.record_settled     (pnl_usd computed)
              BankrollTracker.update         ★ NEW (Kelly feedback loop)
```

The new BankrollTracker is a small in-runner state object that starts at $1,000 (locked decision #10) and updates on every settled PnL. v2.1 reads it via a getter callback. Independent of dashboard's load_bankroll() which derives from JSONL — they converge by construction but the dashboard remains the source of truth for operator-facing display.

3. Class definition — SpotAwareStrategyV2_1

3.1 Constructor signature

```
from __future__ import annotations
from collections.abc import Callable
from polymarket_bot.strategies.spot_aware_v2 import SpotAwareStrategyV2
from polymarket_bot.strategies.kelly_config import KellyConfig
from polymarket_bot.strategies.spot_aware_priors import SpotAwarePriors
from polymarket_bot.backtest.types import Decision, MarketEvent


class SpotAwareStrategyV2_1(SpotAwareStrategyV2):
    name: str = 'spot_aware_v2_1'

    def __init__(
        self,
        *,
        priors: SpotAwarePriors,
        bankroll_getter: Callable[[], float],
        kelly: KellyConfig | None = None,
        multi_fire_cap: int = 2,
        tier_relax_delta: tuple[str, ...] = ('15-30%', '30-50%'),
        ev_bad_bucket: tuple[float, float] = (0.02, 0.05),
        **v2_kwargs: object,
    ) -> None:
        super().__init__(priors=priors, **v2_kwargs)
        self._bankroll_getter = bankroll_getter
        self._kelly = kelly or KellyConfig()
        self._multi_fire_cap = multi_fire_cap
        self._tier_relax = frozenset(tier_relax_delta)
        self._ev_bad_lo, self._ev_bad_hi = ev_bad_bucket
        self._fires_this_window: int = 0
        # _fired_for_window inherited from v2 but overridden behavior
```

3.2 Overridden methods

Method	Override reason
_on_boundary	reset _fires_this_window AND _fired_for_window on window open
_maybe_fire	new filter chain (CH1-CH3); call super; sub Kelly size; rewrite rationale

3.3 _maybe_fire override (exact body)

```
def _maybe_fire(self, now_ts_utc_ns: int) -> Decision | None:
    # Multi-fire gate: stop firing once cap reached for this window.
    if self._fires_this_window >= self._multi_fire_cap:
        return None

    # Pre-compute Δ for the new filters.
    if self._spot_now is None or self._window_open_spot is None:
        return None
    d_bps = delta_bps(self._spot_now, self._window_open_spot)

    # CH1 — reject Δ=0 (no skew → confirmed LOSER, Phase G §4.3).
    if d_bps == 0.0:
        return None

    # CH3 — tier-specific Δ floor.
    # Determine tier from latest best_ask snapshot.
    side_guess: Side = 'UP' if d_bps > 0 else 'DOWN'
    ask_guess = self._best_ask_for_side.get(side_guess)
    if ask_guess is None:
        return None
```

```
    tier = _tier_label(ask_guess)  # '<15%' / '15-30%' / etc.
    min_d = 0.0 if tier in self._tier_relax else self._delta_min
    if abs(d_bps) &lt; min_d:
        return None

    # IMPORTANT: temporarily relax v2's hard self._delta_min for the
    # super() call. We do this by setting an instance flag the parent
    # checks, OR (simpler) by inlining v2's logic. Spec uses inline.
    base_decision = self._maybe_fire_inline(d_bps, side_guess, ask_guess, now_ts_utc_ns)
    if base_decision is None:
        return None

    # CH2 — reject EV ∈ [0.02, 0.05).
    ev = _parse_ev(base_decision.rationale)
    if self._ev_bad_lo &lt;= ev &lt; self._ev_bad_hi:
        # Reset v2's _fired_for_window so the cell can re-evaluate
        # later (mirrors v3's regime-gate reset pattern).
        self._fired_for_window = None
        return None

    # CH5/CH6 — Kelly sizing: replace size_usd from bankroll + p_win + price.
    bankroll = self._bankroll_getter()
    p_win = _parse_p_win(base_decision.rationale)
    price = base_decision.price_limit
    bet = self._kelly.bet_size(bankroll=bankroll, p_win=p_win, price=price)
    if bet &lt; self._kelly.min_bet_abs:
        self._fired_for_window = None
        return None

    # Rewrite rationale (self-tagging; avoids v3 inheritance bug).
    new_rationale = _rewrite_rationale_v2_1(
        base_decision.rationale,
        kelly_f=self._kelly.last_kelly_f,
        bet_usd=bet,
        bankroll_at_fire=bankroll,
        fire_n_in_window=self._fires_this_window + 1,
    )
    self._fires_this_window += 1
    return dc_replace(base_decision, size_usd=bet, rationale=new_rationale)
```

4. KellyConfig dataclass

File: src/polymarket_bot/strategies/kelly_config.py

```
"""Kelly fractional sizing config + bet computation."""
from __future__ import annotations
from dataclasses import dataclass, field

@dataclass(slots=True)
class KellyConfig:
    kelly_mult: float = 0.10        # fractional Kelly (start conservative)
    max_bet_pct: float = 0.10       # never more than 10% of bankroll
    max_bet_abs: float = 500.0      # Polymarket retail liquidity cap
    min_bet_abs: float = 1.00       # below this, drop
    last_kelly_f: float = field(default=0.0, init=False)

    def __post_init__(self) -> None:
        if not 0 &lt; self.kelly_mult &lt;= 1.0:
            raise ValueError(f'kelly_mult must be in (0, 1]; got {self.kelly_mult}')
        if not 0 &lt; self.max_bet_pct &lt;= 0.5:
            raise ValueError(f'max_bet_pct must be in (0, 0.5]; got {self.max_bet_pct}')
        if self.max_bet_abs &lt;= 0:
            raise ValueError(f'max_bet_abs must be &gt; 0; got {self.max_bet_abs}')

    def kelly_f(self, p_win: float, price: float) -> float:
        """Standard Kelly fraction for a binary bet at given price."""
        if price &gt;= 1.0 or price &lt;= 0.0 or p_win &lt;= price:
            return 0.0
        return (p_win - price) / (1.0 - price)

    def bet_size(self, *, bankroll: float, p_win: float, price: float) -> float:
        """Return clamped bet size in USD. Stores last_kelly_f for telemetry."""
        f = self.kelly_f(p_win, price)
        self.last_kelly_f = f
        if f == 0.0 or bankroll &lt;= 0:
            return 0.0
        bet = bankroll * self.kelly_mult * f
        bet = min(bet, bankroll * self.max_bet_pct, self.max_bet_abs)
        return max(self.min_bet_abs, bet) if bet &gt;= self.min_bet_abs else 0.0
```

Why mutable last_kelly_f? It's a write-once-per-call telemetry cache consumed by the rationale builder. Could be returned from bet_size() but the single-return signature keeps callers clean. NOT thread-shared.

5. Bankroll-getter integration

5.1 Why a getter not a static value

Kelly sizing requires LIVE bankroll. The strategy is constructed once at startup; bankroll grows/shrinks as fires settle. Inject a callable so the strategy reads current value at fire time — this is the compounding hook.

5.2 PaperRunner changes (BankrollTracker)

```
# In runner.py, add a small tracker (~30 LOC):
@dataclass(slots=True)
class BankrollTracker:
    starting_usd: float = 1000.0  # locked decision #10
    realized_pnl_usd: float = 0.0
    # Update on every settled record (fire-and-forget; thread-safe via
    # async single-event-loop assumption).
    def current(self) -> float:
        return self.starting_usd + self.realized_pnl_usd
    def apply_settle(self, pnl_usd: float) -> None:
        self.realized_pnl_usd += pnl_usd
    def apply_correction(self, original_pnl: float, corrected_pnl: float) -> None:
        # Phase 15.5 settled_correction overlay support.
        self.realized_pnl_usd += (corrected_pnl - original_pnl)

# PaperRunner.__init__ adds:
#   self._bankroll = BankrollTracker(starting_usd=PMB_STARTING_BANKROLL or 1000)
# Then exposes self._bankroll.current as the getter passed to strategy.
```

5.3 Startup recovery (cross-restart consistency)

On startup, BankrollTracker reads recent settled records from state/decisions/*.jsonl and replays them to compute the current bankroll. Avoids the 'restart resets the compound clock' bug. Mirrors the orphan-fire-recovery pattern already in PaperRunner.

Tests must cover: empty log = starting_usd · log with N settles = starting + Σ pnl · log with corrections = starting + Σ corrected_pnl · cross-day boundary handling.

6. Multi-fire — PaperRunner changes required

Critical finding from reading runner.py: the singular pending_decision invariant at _on_decision blocks multi-fire at the runner level. v2.1 cannot enable multi-fire by itself.

6.1 Current invariant (runner.py:448)

```
if self._state.pending_decision is not None or self._state.settled_fill is not None:
    LOG.warning('second decision in window %s; dropping', self._state.window_ts)
    return  # BLOCKED
```

6.2 v2.1-compatible state change

```
# Refactor _state to support multiple pending decisions per window:
@dataclass
class _RunnerState:
    window_ts: int | None = None
    pending_decisions: list[PendingDecision] = field(default_factory=list)
    multi_fire_cap: int = 1  # v2 keeps 1; v2.1 sets to 2 from env

    @property
    def pending_decision(self) -> Decision | None:
        # Backwards-compat read for v1/v2/v3 code paths
        return self.pending_decisions[0].decision if self.pending_decisions else None

@dataclass
class PendingDecision:
    decision: Decision
    decision_id: str
    side: Side
    fill: Fill | None  # populated by ShadowBook
    settled_fill: Fill | None = None
```

6.3 _on_decision guard relaxation

```
if len(self._state.pending_decisions) &gt;= self._state.multi_fire_cap:
    LOG.info(
        'fire %d/%d in window %s; capping',
        len(self._state.pending_decisions), self._state.multi_fire_cap,
        self._state.window_ts,
    )
    return
# ... otherwise proceed as before, just append rather than assign:
self._state.pending_decisions.append(PendingDecision(decision, decision_id, side, fill))
```

6.4 Settle loop changes

_settle_current_window must iterate all pending_decisions, not just one. Each settles independently (may have different asset_id and outcome). BankrollTracker accumulates each settle's pnl_usd.

6.5 Per-asset_id NOT per-side (incident defense)

Per **project_per_side_state_pollution_pattern.md**: strategies that key book state by event.side instead of event.asset_id read state from the WRONG window's token (warmer pre-fetches forward windows). v2.1 must NOT regress this. Its multi-fire counter is keyed by window_ts, not by side. The PendingDecision struct carries asset_id explicitly.

7. Rationale + decision-log schema

7.1 Rationale format (self-tagging)

> **Past incident:** v3 inherited v2's rationale string unchanged, causing downstream tooling (audit, watcher) to mis-attribute v3 fires as v2 fires. Fix in commit 74f7541 was to rewrite rationale at fire time. v2.1 follows the same pattern.

v2 rationale (from spot_aware_v2.py:285):

```
strategy=spot_aware_v2 mode=tier_restricted delta_bps=+5.02 p_win=0.400 ev=+0.1650 ttl=30s side
=UP
```

v2.1 rationale (new format):

```
strategy=spot_aware_v2_1 mode=kelly_multi delta_bps=+5.02 p_win=0.400 ev=+0.1650 ttl=30s side=U
P kelly_f=0.357 bet_usd=89.50 bankroll=1006.01 fire_n=1/2
```

Rewrite function (~15 LOC): regex-sub strategy= and mode= tokens, then append the new telemetry tokens. Word-boundary regex prevents accidental rewrites of substrings.

7.2 Decision-log JSONL additions

Add these structured fields to the `fired` record so downstream tools don't have to parse rationale:

Field	Type	Source	Backward-compat
kelly_f	float	KellyConfig.last_kelly_f	absent = legacy fire
bet_usd	float	Decision.size_usd post-Kelly	fallback to size_usd
bankroll_at_fire	float	BankrollTracker.current()	absent = legacy
fire_n_in_window	int	1-indexed (1 or 2 for v2.1)	absent = legacy=1
multi_fire_cap	int	from env	absent = legacy=1

scripts/four_day_audit.py and scripts/watch_first_v3_fire.py need no changes — they read rationale or kind=fired without these new fields.

8. Dashboard integration

Outcome of v2.1 shipping: the v3 dashboard (FastAPI + React, port 8505) reflects v2.1 activity within seconds of the first fire.

8.1 Strategy badge (header)

The badge currently reads SPOT_AWARE (parsed from /api/state.strategy). It will update to SPOT_AWARE_V2_1 automatically — the dashboard reads from decision rationale via existing parser. No dashboard code change needed.

8.2 Phase label (stale 'PHASE 11.A.2')

The dashboard header still shows 'PHASE 11.A.2' (hardcoded in React). Phase H ships an update to display 'PHASE H' or 'PHASE H v2.1'. This is a 1-line change in the dashboard frontend, ride-along with H.5 deploy.

8.3 Decision tape (auto-updates)

Each v2.1 fire appears in the decision tape with side/tier/limit. Multi-fire means 2 entries per window. Operator sees this immediately.

8.4 New metric panels (Phase H follow-up)

Panel	Source field	Display
Avg Kelly bet (24h)	rationale.kelly_f	mean × current bankroll
Fires per window distribution	fire_n_in_window	histogram: 1 vs 2
Compound factor	bankroll/1000	X.XXx since start
Worst drawdown (rolling)	bankroll min vs max in window	DD %

These can ship in Phase I (dashboard observability) — not blocking v2.1 deploy.

8.5 Cumulative PnL curve continuity

v2.1's settled records flow through the same load_bankroll() pipeline. The cumulative PnL curve continues smoothly from current ~-$19 baseline. If v2.1 hits its Conservative trajectory, the curve bends sharply upward within days. Visible on /api/state.cumulative_pnl_series.

9. Test plan — 18 tests

9.1 KellyConfig (5 tests) — tests/strategies/test_kelly_config.py

Test	Setup → Assertion
test_kelly_f_zero_when_no_edge	p_win=0.30, price=0.30 → 0.0
test_kelly_f_classic	p_win=0.55, price=0.30 → 0.357 (±1e-6)
test_bet_clamps_to_max_abs	bankroll=10000, f=0.4 → bet=500
test_bet_clamps_to_max_pct	bankroll=1000, f=0.5 → bet=100 (10%)
test_bet_below_min_returns_zero	bankroll=100, f=0.01 → bet=0 (below $1 floor)

9.2 v2.1 filters (6 tests) — tests/strategies/test_spot_aware_v2_1.py

Test	Setup → Assertion
test_v2_1_name_marker	strategy.name == 'spot_aware_v2_1'
test_v2_1_rejects_delta_zero	Δ=0 → on_event returns None
test_v2_1_rejects_ev_in_bad_bucket	EV ∈ [0.02, 0.05) → None
test_v2_1_allows_30_50_with_low_delta	tier 30-50%, |Δ|=2 → Decision
test_v2_1_requires_5bps_for_70_85	tier 70-85%, |Δ|=2 → None
test_v2_1_inherits_tier_50_70_reject	tier 50-70% → None (v2 inheritance)

9.3 v2.1 Kelly + rationale (4 tests)

Test	Setup → Assertion
test_v2_1_kelly_sizes_replace_default	Decision.size_usd ≠ DEFAULT (replaced)
test_v2_1_rationale_self_tags	'strategy=spot_aware_v2_1' in rationale
test_v2_1_rationale_includes_kelly_metadata	kelly_f= bet_usd= bankroll= fire_n= present
test_v2_1_no_fire_when_bankroll_too_low	bankroll=$1 → bet below min_bet → None

9.4 Multi-fire (3 tests)

Test	Setup → Assertion
test_multi_fire_two_per_window	drive two qualifying ticks → 2 Decisions
test_multi_fire_blocked_at_cap	drive third tick same window → None
test_multi_fire_resets_on_window_boundary	WindowBoundary open → counter reset

9.5 BankrollTracker (no strategy) — tests/paper/test_bankroll_tracker.py

Test	Setup → Assertion
test_bankroll_starts_at_decision_10_default	BankrollTracker() → current() == 1000
test_bankroll_applies_settle	apply_settle(+5) → current() == 1005
test_bankroll_applies_correction	apply_correction(orig=2, corr=-3) → delta=-5
test_bankroll_recovers_from_jsonl	feed N settled records → current matches sum

Total: 22 tests. The H.1 PDF said ~15; actual count is 22 once BankrollTracker is broken out as a separate testable unit.

10. .envrc spec

10.1 Lines to add (above PHASE-F markers)

```
# PHASE-H 2026-05-13: v2.1 deploy — Kelly + multi-fire + filter changes
# Rollback: cp .envrc.phase-H-pre-v2_1-backup-YYYYMMDD-HHMMSS .envrc
# export STRATEGY=spot_aware_v2  (commented; was the PHASE-F-4 setting)
export STRATEGY=spot_aware_v2_1
export KELLY_MULT=0.10
export MULTI_FIRE_CAP=2
export PMB_STARTING_BANKROLL=1006.01  # current actual bankroll
```

10.2 Backup convention

```
BACKUP=.envrc.phase-H-pre-v2_1-backup-$(date +%Y%m%d-%H%M%S)
cp .envrc $BACKUP
```

11. scripts/run_paper.py dispatch addition

```
# In _build_strategy, after the spot_aware_v3 branch:
if name == 'spot_aware_v2_1':
    from polymarket_bot.strategies.spot_aware_v2_1 import SpotAwareStrategyV2_1
    from polymarket_bot.strategies.kelly_config import KellyConfig
    kelly = KellyConfig(
        kelly_mult=_env_float('KELLY_MULT', 0.10) or 0.10,
        max_bet_abs=_env_float('KELLY_MAX_BET_ABS', 500.0) or 500.0,
    )
    multi_fire_cap = int(_env_float('MULTI_FIRE_CAP', 2) or 2)
    # bankroll_getter wired in main() after PaperRunner constructed
    return SpotAwareStrategyV2_1(
        priors=load_priors(),
        bankroll_getter=lambda: 1000.0,  # PLACEHOLDER — rebound in main()
        kelly=kelly,
        multi_fire_cap=multi_fire_cap,
    )

# In main(), after PaperRunner is constructed:
if isinstance(strategy, SpotAwareStrategyV2_1):
    strategy._bankroll_getter = runner._bankroll.current
```

12. Deployment runbook (H.5)

Exact SSH command sequence (model on PHASE-F-4 today):

```
ssh -i $HOME/Downloads/LightsailDefaultKey-eu-west-1.pem -p 2222 ops@108.132.51.116 'set -e
cd /home/ops/polymarket-5m-bot

# 0. Pre-pull verify
git rev-parse --short HEAD
systemctl is-active bot-staging.service

# 1. Pull latest with v2.1 code
git pull --ff-only origin claude/jovial-hermann-7e103f
git rev-parse --short HEAD  # confirm expected commit

# 2. Backup .envrc (per feedback_vps_deploy_verify_late memory)
BACKUP=.envrc.phase-H-pre-v2_1-backup-$(date +%Y%m%d-%H%M%S)
cp .envrc $BACKUP
echo "Backup: $BACKUP"

# 3. Apply .envrc edits via sed (atomic; preserves comments)
sed -i.tmp -E "s|^export STRATEGY=spot_aware_v2\$|# PHASE-H 2026-05-13: v2.1 deploy\n# export S
TRATEGY=spot_aware_v2\nexport STRATEGY=spot_aware_v2_1\nexport KELLY_MULT=0.10\nexport MULTI_FI
RE_CAP=2\nexport PMB_STARTING_BANKROLL=1006.01|" .envrc
rm -f .envrc.tmp

# 4. Capture DEPLOY_TS_5 (per feedback_vps_timestamp_arithmetic memory —
#    NEVER compute from systemctl text; ALWAYS read $(date +%s) on VPS)
DEPLOY_TS_5=$(date +%s)
echo "DEPLOY_TS_UNIX_5=${DEPLOY_TS_5}"

# 5. Restart
sudo systemctl restart bot-staging.service
sleep 3
systemctl show bot-staging.service -p MainPID,ActiveEnterTimestamp,NRestarts,ActiveState

# 6. Immediate sanity (within 5s of restart)
systemctl is-active bot-staging.service
journalctl -u bot-staging.service --since "@${DEPLOY_TS_5}" --no-pager | grep -E "(active strat
egy|kelly|multi.fire|bankroll)" | head -5
'
```

13. Verification gates

When	What to check	Pass criteria	Failure →
+5 min	systemctl active, no errors, startup log has 'active strategy: spot_aware_v2_1'	active	ROLLBACK
+5 min	first fire log line has kelly_f= and bet_usd= and rationale=	rationale non-empty	investigate rationale rewrite
+30 min	no queue_overflow / dropped_records log lines; 0 Anthropic calls = 0	0 / 0	investigate I/O backpressure
+30 min	bankroll tracker matches dashboard /api/state.bankroll	exact match (±$0.50)	BankrollTracker bug
+24 h	fires/day in range [16, 50]; multi-fire windows present	present (fire_n=2 records)	EXTEND or investigate
+24 h	WR ≥ 47% on settled n ≥ 30 (early signal)	WR ≥ 47%	EXTEND
+24 h	daily PnL within [-$15, +$50]	no extreme	ROLLBACK if < -$30
+7 d	Soak gate per Section 14	see Section 14	per Section 14

14. Past-incident defense matrix

Every confirmed bug class from the bot's history mapped to v2.1's specific defense:

Past incident	How v2.1 avoids it
v3 rationale-inheritance (today)	v2.1 rewrites rationale at fire time, self-tags 'strategy=spot_aware_v2_1'
per-side state pollution (16.B.4)	v2.1 tracks _fires_this_window by window_ts; PendingDecision carries asset_id
LLM queue overflow (2026-05-10)	v2.1 has NO async-await on external API in fire path (no LLM)
Instant-settle queue overflow (5/9)	v2.1 adds no inline awaits; BankrollTracker.apply_settle is sync
FD orphan dashboard incident (5/9)	v2.1 doesn't write to decision log; PaperRunner's existing inode-stable append unchanged
Phantom-fill (Phase 11.A)	v2.1 inherits v2's tier rejection; settled_correction overlay continues to work
Chainlink-vs-polymarket resolve (5/10)	v2.1 doesn't touch resolution; PaperRunner's gamma retry + correction overlay unchanged
Wide_fire ensemble negative selection (5/11)	v2.1 doesn't use LLM ensemble; Phase F cut intact
Cross-restart double-fire (5/10)	v2.1 multi-fire CAP scoped to single window_ts; restart resets _fires_this_window
VPS timestamp drift (5/12)	Deployment runbook uses VPS $(date +%s); no mental arithmetic
Phase 15.5 truth-pass break	settled_correction continues to work; BankrollTracker.apply_correction wired
pending_decision singular invariant	_RunnerState refactor to pending_decisions: list; backward-compat property
OSError on disk-full kills runner	Existing try/except in _on_decision preserved
Aggressive-mode high variance	Kelly bet caps + ROLLBACK gate at WR<45% over 48h

15. Rollback decision tree

15.1 ROLLBACK now (within H.5 hour)

Triggers: service fails to start, startup log errors, queue overflow, rationale missing self-tag, bankroll-getter raises.

```
ssh ... 'cd /home/ops/polymarket-5m-bot &amp;&amp; \
  cp .envrc.phase-H-pre-v2_1-backup-YYYYMMDD-HHMMSS .envrc &amp;&amp; \
  sudo systemctl restart bot-staging.service'
```

15.2 ROLLBACK over soak (H.6 trigger)

Triggers (any of):
• WR < 45% over 48h continuous
• daily_pnl < -$30 on any single day
• bankroll drops below $500 (50% drawdown from start)
• 3 consecutive losing days (Bonferroni-corrected $\alpha \approx 1.5\%$)

15.3 EXTEND (re-soak)

Triggers: WR in [45%, 52%), or fires/day in [10, 30), or daily_pnl in [-30, -10) on worst day. Action: keep v2.1 running another 7 days, re-audit.

15.4 PROMOTE

Triggers (all of):
• WR ≥ 52% on n ≥ 100 settled fires
• daily_pnl_min ≥ -$10 over the 7-day window
• fires/day ≥ 30 (multi-fire 2× expected)
• queue_overflow_incidents == 0
• bankroll ≥ $1,500 (50% gain) — sanity check

Action: schedule Phase I (live trading prep / readiness review).

16. Open questions for operator (Q1-Q5)

Q	Question	Options	Default (this spec)
Q1	Hour-of-day gate (S1)?	YES skip 5/14/23 UTC · NO no gate	NO (sample too thin)
Q2	Regime gate (W3)?	A no gate · B half-Kelly on chop · C trending only	A no gate (start simple)
Q3	Multi-fire cap?	2 / 3 per window	2 (conservative)
Q4	Kelly multiplier?	0.10 / 0.25	0.10 (smoother)
Q5	Live-trading promotion threshold?	$2k / $5k / $10k	$2k (locked decision #12)

If operator approves all defaults: ship code as specified. If operator overrides any: only env vars change (KELLY_MULT, MULTI_FIRE_CAP). Code is parameterized; no logic changes needed.

17. Honest pushback — engineering risks

17.1 Code paths that need careful review

• **BankrollTracker recovery from JSONL on startup**: must handle settled_correction records (Phase 15.5 overlay). Test coverage critical.
• **PendingDecision list refactor in PaperRunner**: pending_decision property must keep v1/v2 compatibility. Run existing v2 test suite as regression.
• **Multi-fire settle ordering**: if window 1's 2 fires settle out of order, BankrollTracker must still converge. Test with synthetic settle order.

17.2 Race conditions to test under load

Per **feedback_test_under_load**: async I/O in hot path needs a queue-doesn't-overflow test. v2.1 adds no async I/O (Kelly is sync), but multi-fire DOUBLES fire-rate and thus DOUBLES log-write rate. Verify DecisionLog write throughput at 50 fires/day (no concern), and confirm no compound effect.

17.3 What we DON'T know yet

• Does the v_next 30-50%×|Δ|<5 cell generalize beyond the 5-day backtest? Phase G honest-pushback covers this. v2.1's ROLLBACK gate is the safety net.
• Does Polymarket retail liquidity actually support $500 bets at 30-50% tier prices? May be lower. Monitor fill rate; if it drops, lower max_bet_abs.
• Does multi-fire 2× actually mean 2× fires/day, or do the signals correlate and the second fire is rare? First 24h of data will tell.

17.4 The big risk

> **Kelly compounding amplifies BOTH expected return AND variance.** Phase G Scenario C showed 97% of paths hit catastrophic drawdown at 50% WR (regression). If the v_next backtest WR (58%) was sample bias, v2.1 with Kelly 0.10× × multi-fire 2× may bleed bankroll fast.

Mitigations baked in: 0.10× (not 0.25×) Kelly, $500 absolute bet cap, multi-fire 2× (not 3×), ROLLBACK at WR<45% over 48h. Combined: max ~6.4% of bankroll deployed per window (2 × 0.10 × 0.357 max-edge). Sane.

18. Sign-off checklist (operator → assistant)

Before Phase H.2 code work begins, operator confirms:

[ ] Section 3 — class definition acceptable
[ ] Section 4 — KellyConfig fields + defaults OK
[ ] Section 5 — BankrollTracker in PaperRunner OK
[ ] Section 6 — multi-fire PaperRunner refactor OK
[ ] Section 7 — rationale format + new JSONL fields OK
[ ] Section 9 — 22-test surface OK
[ ] Q1: NO hour gate · Q2: NO regime gate · Q3: cap=2 · Q4: Kelly 0.10× · Q5: $2k promote
[ ] Section 14 — past-incident defense matrix understood
[ ] Section 15 — ROLLBACK criteria acceptable

Sign-off via: reply 'go phase-H.2' OR 'go with overrides: Q1=YES Q3=3 ...'

End of Phase H final implementation spec · 2026-05-13 · 18 sections · 22 tests · 14 past-incident defenses · ~30 pages

Source prompt: phase-prompts/phase-H-FINAL-spec-2026-05-13.txt
Companion docs: PHASE_G_EDGE_DISCOVERY.pdf, PHASE_H_v2_1_IMPLEMENTATION.pdf